ExplainX-CXR: Explainable Chest X-ray Abnormality Classifier

Reason: medical imaging projects combine high societal impact, rigorous evaluation (sensitivity/specificity, ROC), and immediate technical depth: transfer learning on dense backbones (EfficientNet/ResNet), domain-specific preprocessing (lung cropping, contrast enhancement), class-imbalance mitigation, uncertainty estimation, explainability (Grad-CAM), and cautious deployment (model cards, clinical disclaimers). This project demonstrates both applied ML competence **and** the ability to address data ethics and safety — which stands out in interviews and on GitHub for data-science roles in healthcare, startups, and R&D teams. You can realistically complete it on Colab, show clear metrics (ROC AUC, sensitivity), produce a Streamlit demo and a well-documented repo — an ideal showcase for entry-level data scientist roles.

Great — below is a **detailed, ready-to-run 8-week project plan** for **ExplainX-CXR: Explainable Chest X-ray Abnormality Classifier**. It contains weekly milestones, an exact Colab notebook **cell-by-cell outline** (with runnable code snippets), key hyperparameters, baseline training/evaluation loops, Grad-CAM explainability, deployment notes, and suggested experiment grid. This is optimized for a single-GPU Colab (T4/P100) and public datasets (Kaggle Chest X-ray Pneumonia or NIH subsets).

8-Week Project Plan — ExplainX-CXR

Overview / goals

- Build a transfer-learning CNN classifier that predicts **pneumonia / normal** from chest X-rays.
- Provide **explainability** using Grad-CAM, and **uncertainty** via MC-Dropout.
- Deliverables: reproducible Colab notebook, trained model weights, evaluation report (ROC, sensitivity/specificity, calibration), Grad-CAM visualizer, Streamlit demo, model card & README.

Weekly milestones (8 weeks, part-time 10–15 hours/week)

Week 1 — Project setup & data ingestion

- Set up repo, Colab skeleton, Kaggle access, download dataset (Kermany chest X-ray or NIH subsets), exploratory data analysis (counts, sample images).

- Deliverable: Colab notebook with data download and EDA.

Week 2 — Preprocessing & dataset pipeline

- Implement consistent preprocessing (resize, lung-ROI optional), PyTorch `Dataset` and `DataLoader`, augmentation pipeline.
- Deliverable: working data pipeline and train/val split (stratified).

Week 3 — Baseline model & training loop

- Implement transfer learning baseline (EfficientNet-B0 or ResNet34), training loop, logging (TensorBoard / wandb optional), checkpointing.
- Deliverable: first baseline model and training curves.

Week 4 — Evaluation & metrics

- Add ROC AUC, confusion matrix, sensitivity, specificity, per-class F1, calibration plots. Tune class weights / sampling to handle imbalance.
- Deliverable: evaluation notebook section + improved model.

Week 5 — Explainability & uncertainty

- Integrate Grad-CAM visualizations, sanity checks on explanations. Add MC-Dropout for uncertainty and calibration.
- Deliverable: Grad-CAM visual examples and uncertainty outputs.

Week 6 — Model improvements & ablation

- Try augmentations, optimizer variants, learning-rate scheduling, input size changes. Add small segmentation/ROI head if desired.
- Deliverable: ablation results (table) and chosen final model.

Week 7 — Deployment & demo

- Build a Streamlit demo that accepts uploaded X-ray and shows prediction + Grad-CAM + uncertainty. Containerize (Dockerfile).
- Deliverable: deployed demo (local/Heroku/Cloud Run) or instructions.

Week 8 — Report, README, model card & polish

- Write final report, model card (intended use, limitations), prepare GitHub README with instructions and reproducible runtimes. Final testing and small dataset of inference examples.
- Deliverable: GitHub repo ready; slides or video demo.

Exact Colab Notebook — Cell-by-cell outline (PyTorch)

> Use this as the sequence of notebook cells. Copy/paste cells into Colab and run sequentially. Replace paths and secrets where indicated.

Cell 1 — Notebook title & runtime note

```
# ExplainX-CXR: Explainable Chest X-ray Classifier (Colab)
# Runtime: GPU required (Colab T4 / P100 recommended)
# Author: <Your Name> — Week-by-week plan linked in repo
```

Cell 2 — Install dependencies

```
# Cell: Install required libs (run once)
!pip install -q timm==0.9.2 pytorch-lightning==1.8.6 grad-cam==1.4.6 albumentations==1.3.0 kaggle
```

Notes:

- `timm` provides EfficientNet and many backbones.
- `grad-cam` is the pytorch-grad-cam package.
- `pytorch-lightning` optional — you can copy training loop code below if you prefer raw PyTorch.

Cell 3 — Imports

```
# Cell: Imports
import os, random, math, shutil, json, time
from pathlib import Path
import numpy as np
import pandas as pd
from tqdm.notebook import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, roc_curve, accuracy_score, f1_score, confusion_matrix, precision_recall_fscore_support
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models as models
import timm
from PIL import Image
import matplotlib.pyplot as plt
import albumentations as A
from albumentations.pytorch import ToTensorV2

# Grad-CAM
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
```

Cell 4 — Set seeds & device

```
# Cell: reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

Cell 5 — Dataset download instructions (Kaggle)

```
# If using Kaggle dataset (Chest X-ray Pneumonia by Kermany):
# 1) Upload kaggle.json (API token) to Colab: use Files sidebar or below code.
# 2) Run the command below to download.

# NOTE: If you prefer NIH ChestX-ray14, replace with its fetching instructions.

# Cell: download dataset (Kaggle)
from google.colab import files
# Run once: upload your kaggle.json
# files.upload()
```

```
# Then:
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
!unzip -q chest-xray-pneumonia.zip -d data
```

(If user uploads dataset manually, place it under `data/` with structure `data/chest_xray/train/PNEUMONIA`, `data/chest_xray/train/NORMAL`, etc.)

Cell 6 — Inspect file structure & counts (EDA)

```
# Cell: EDA - show counts and samples
base = Path('data/chest_xray')
for split in ['train','val','test']:
    p = base/split
    if p.exists():
        print(split, "->", {cls: len(list((p/cls).glob('*.jpeg'))) for cls in os.listdir(p)})
    else:
        print(split, "not found")
```

If val doesn't exist, create stratified train/val split in next cell.

Cell 7 — Create stratified train/val split (if needed)

```
# Build a dataframe of filepaths + labels and split
root = Path('data/chest_xray/train')
rows = []
for label in ['PNEUMONIA','NORMAL']:
    for img in (root/label).glob('*.jpeg'):
        rows.append({'filepath': str(img), 'label': label})
df = pd.DataFrame(rows)
df['y'] = (df['label']=='PNEUMONIA').astype(int)
train_df, val_df = train_test_split(df, test_size=0.15, stratify=df['y'], random_state=SEED)
print(len(train_df), len(val_df))
```

Cell 8 — Quick EDA visualizations

```
# show sample images
def show_samples(df, n=6):
    fig, axs = plt.subplots(1, n, figsize=(15,3))
    for i, p in enumerate(df['filepath'].sample(n)):
        img = Image.open(p).convert('RGB')
        axs[i].imshow(img)
        axs[i].axis('off')

show_samples(df[df['label']=='PNEUMONIA'])
show_samples(df[df['label']=='NORMAL'])
```

Cell 9 — Preprocessing & Augmentations (Albumentations)

```
# Image size and augmentation params
IMG_SIZE = 224

train_transforms = A.Compose([
    A.RandomResizedCrop(IMG_SIZE, IMG_SIZE, scale=(0.8,1.0)),
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=10, p=0.3),
    A.OneOf([A.RandomBrightnessContrast(p=0.5), A.CLAHE(p=0.5)], p=0.5),
    A.Normalize(mean=(0.485,0.456,0.406), std=(0.229,0.224,0.225)),
    ToTensorV2(),
])

val_transforms = A.Compose([
    A.Resize(IMG_SIZE, IMG_SIZE),
    A.Normalize(mean=(0.485,0.456,0.406), std=(0.229,0.224,0.225)),
    ToTensorV2(),
])
```

Notes:

- We use ImageNet normalization for transfer learning.

Cell 10 — PyTorch Dataset class

```
class CXRDataset(Dataset):
    def __init__(self, df, transforms=None):
        super().__init__()
        self.df = df.reset_index(drop=True)
```

```
        self.transforms = transforms

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = np.array(Image.open(row['filepath']).convert('RGB'))
        if self.transforms:
            augmented = self.transforms(image=img)
            img = augmented['image']
        label = torch.tensor(row['y'], dtype=torch.float32)  # binary
        return img, label
```

Cell 11 — Dataloaders

```
BATCH_SIZE = 16  # reduce to 8 if OOM on Colab

train_ds = CXRDataset(train_df, transforms=train_transforms)
val_ds = CXRDataset(val_df, transforms=val_transforms)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=2,
pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2,
pin_memory=True)
```

Cell 12 — Model: EfficientNet-B0 (timm) with custom head

```
# Use timm to instantiate backbone
import timm

def build_model(model_name='efficientnet_b0', pretrained=True, n_classes=1):
    backbone = timm.create_model(model_name, pretrained=pretrained, num_classes=0,
global_pool='avg')
    n_features = backbone.num_features
    head = nn.Sequential(
        nn.Linear(n_features, 512),
        nn.ReLU(),
        nn.Dropout(0.4),
        nn.Linear(512, n_classes)
    )
```

```
    model = nn.Sequential(backbone, head)
    return model

model = build_model('efficientnet_b0', pretrained=True).to(device)
print(model)
```

Notes:

- Output is a single logit; use `BCEWithLogitsLoss` for training.

Cell 13 — Loss, optimizer, scheduler, metrics

```
criterion = nn.BCEWithLogitsLoss()  # for binary
LR = 1e-4
WEIGHT_DECAY = 1e-4
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5,
patience=2, verbose=True)

# Training hyperparams
EPOCHS = 25
GRAD_CLIP_NORM = 1.0
```

Key hyperparameters (baseline):

- batch_size = 16
- image_size = 224
- lr = 1e-4 (AdamW)
- weight_decay = 1e-4
- epochs = 25–30
- scheduler: ReduceLROnPlateau or CosineAnnealingLR
- early stopping patience = 5

Cell 14 — Utility: save & load checkpoints

```
def save_checkpoint(state, filename='checkpoint.pth'):
    torch.save(state, filename)

def load_checkpoint(path, model, optimizer=None, map_location=device):
    ckpt = torch.load(path, map_location=map_location)
```

```
    model.load_state_dict(ckpt['model_state'])
    if optimizer and 'optim_state' in ckpt:
        optimizer.load_state_dict(ckpt['optim_state'])
    return ckpt
```

Cell 15 — Training & validation loops (baseline)

```
from sklearn.metrics import roc_auc_score

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0
    all_targets = []
    all_preds = []
    for imgs, labels in tqdm(loader):
        imgs = imgs.to(device)
        labels = labels.to(device).unsqueeze(1)
        optimizer.zero_grad()
        outputs = model(imgs)  # logits
        loss = criterion(outputs, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP_NORM)
        optimizer.step()
        running_loss += loss.item() * imgs.size(0)
        all_targets.append(labels.detach().cpu().numpy())
        all_preds.append(torch.sigmoid(outputs).detach().cpu().numpy())
    avg_loss = running_loss / len(loader.dataset)
    targets = np.vstack(all_targets)
    preds = np.vstack(all_preds)
    try:
        auc = roc_auc_score(targets, preds)
    except ValueError:
        auc = float('nan')
    return avg_loss, auc

def validate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    all_targets = []
    all_preds = []
    with torch.no_grad():
        for imgs, labels in tqdm(loader):
            imgs = imgs.to(device)
```

```
            labels = labels.to(device).unsqueeze(1)
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * imgs.size(0)
            all_targets.append(labels.cpu().numpy())
            all_preds.append(torch.sigmoid(outputs).cpu().numpy())
    avg_loss = running_loss / len(loader.dataset)
    targets = np.vstack(all_targets)
    preds = np.vstack(all_preds)
    auc = roc_auc_score(targets, preds)
    return avg_loss, auc, targets, preds
```

Cell 16 — Full training loop with checkpointing & early stopping

```
best_val_auc = 0.0
patience = 5
patience_counter = 0

for epoch in range(1, EPOCHS+1):
    train_loss, train_auc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, val_auc, val_targets, val_preds = validate(model, val_loader, criterion, device)
    print(f"Epoch {epoch}: Train loss {train_loss:.4f}, AUC {train_auc:.4f} | Val loss {val_loss:.4f},
AUC {val_auc:.4f}")

    # Scheduler step (ReduceLROnPlateau uses val_metric)
    scheduler.step(val_auc)

    # Save best
    if val_auc > best_val_auc:
        best_val_auc = val_auc
        save_checkpoint({'epoch': epoch, 'model_state': model.state_dict(), 'optim_state':
optimizer.state_dict(), 'val_auc': val_auc}, filename='best_model.pth')
        patience_counter = 0
    else:
        patience_counter += 1
    if patience_counter >= patience:
        print("Early stopping triggered")
        break
```

Baseline expectation: this gives you a decent model quickly. Adjust `BATCH_SIZE` down if OOM.

Cell 17 — Evaluation: thresholding, confusion matrix, sensitivity/specificity

```
# Load best model
ckpt = torch.load('best_model.pth', map_location=device)
model.load_state_dict(ckpt['model_state'])
model.eval()

# Get predictions on validation/test set
val_loss, val_auc, targets, preds = validate(model, val_loader, criterion, device)
preds_bin = (preds >= 0.5).astype(int)
tn, fp, fn, tp = confusion_matrix(targets, preds_bin).ravel()
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)
print(f"Val AUC: {val_auc:.4f}, Sensitivity: {sensitivity:.4f}, Specificity: {specificity:.4f}")
```

Cell 18 — Grad-CAM explainability (examples)

```
# Use pytorch-grad-cam to compute Grad-CAM for one image
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import BinaryClassificationOutputTarget
from pytorch_grad_cam.utils.image import preprocess_image

# Choose target layer (backbone last conv)
# For timm models, find last conv via model[0].conv_head or check model[0].conv_stem...
target_layer = model[0].conv_head if hasattr(model[0],'conv_head') else model[0].conv_stem

cam = GradCAM(model=model, target_layer=target_layer, use_cuda=torch.cuda.is_available())

# Pick a sample image
img_path = val_df['filepath'].sample(1).values[0]
img = np.array(Image.open(img_path).convert('RGB').resize((IMG_SIZE, IMG_SIZE))) / 255.0
input_tensor =
val_transforms(image=(img*255).astype('uint8'))['image'].unsqueeze(0).to(device)

targets = [BinaryClassificationOutputTarget(1)]  # focus on pneumonia class
grayscale_cam = cam(input_tensor=input_tensor, targets=targets)[0]
visualization = show_cam_on_image(img, grayscale_cam, use_rgb=True)

plt.figure(figsize=(6,6))
plt.imshow(visualization)
plt.axis('off')
```

Notes:

- Confirm `target_layer` by inspecting the backbone architecture. Timms often uses `model.forward_features()`.

Cell 19 — Uncertainty: MC-Dropout (simple)

```
# Turn on dropout at inference by setting model.train() for MC samples on dropout layers only
def enable_dropout(m):
    if isinstance(m, nn.Dropout):
        m.train()

def mc_dropout_predict(model, img_tensor, T=20):
    model.eval()
    preds = []
    for _ in range(T):
        model.apply(enable_dropout)  # enable dropout
        with torch.no_grad():
            out = torch.sigmoid(model(img_tensor))
            preds.append(out.cpu().numpy())
    preds = np.vstack(preds)
    mean = preds.mean(axis=0)
    std = preds.std(axis=0)
    return mean, std

# Example on one image
mean, std = mc_dropout_predict(model, input_tensor, T=30)
print("Pred mean:", mean, "std:", std)
```

Caveat: more principled Bayesian methods exist; MC-Dropout is a simple baseline.

Cell 20 — Export model & inference wrapper

```
# Save torchscript (optional)
model.eval()
example = torch.randn(1,3,IMG_SIZE,IMG_SIZE).to(device)
traced = torch.jit.trace(model, example)
traced.save('model_traced.pt')

# Simple inference wrapper
def predict_image(path):
```

```
    img = np.array(Image.open(path).convert('RGB'))
    x = val_transforms(image=img)['image'].unsqueeze(0).to(device)
    with torch.no_grad():
        logit = model(x)
        prob = torch.sigmoid(logit).item()
    return prob
```

Cell 21 — Streamlit demo skeleton (local) — saving files for deployment

```
# Create a file streamlit_app.py with the following content
%%bash
cat > streamlit_app.py <<'PY'
import streamlit as st
from PIL import Image
import torch
import numpy as np
# load model (ensure model file in same dir)
model = torch.jit.load('model_traced.pt', map_location='cpu')
model.eval()
st.title("ExplainX-CXR: Pneumonia Detector (Demo)")
uploaded = st.file_uploader("Upload chest X-ray (JPEG/PNG)", type=['jpg','jpeg','png'])
if uploaded:
    img = Image.open(uploaded).convert('RGB')
    st.image(img, caption='Uploaded image', use_column_width=True)
    # TODO: preprocess same as notebook and inference
    st.text("Prediction pipeline running...")
PY
```

Run demo locally with `streamlit run streamlit_app.py`. For cloud, create Dockerfile and deploy to Cloud Run.

Cell 22 — Model card & ethics notes (markdown)

```
# Model Card (short)
- Model name: ExplainX-CXR EfficientNet-B0 (binary pneumonia)
- Intended use: Assistive triage / research only — NOT a clinical decision tool.
- Data: Kermany Chest X-ray (pediatric) — biases: age range, imaging device, limited geography.
- Metrics: AUC, sensitivity, specificity (report exact values here).
- Limitations: Does not replace clinician judgement; sensitivity/specificity vary by population; possible confounders (hospital markings, labels).
```

- Privacy: Use only de-identified images. Do not store PHI.

Key hyperparameters & suggested search grid

Baseline (start here):

- backbone: `efficientnet_b0` (timm)
- image_size: `224`
- batch_size: `16` (or 8 if low memory)
- lr: `1e-4` (AdamW)
- weight_decay: `1e-4`
- epochs: `25` (early stop patience `5`)
- optimizer: `AdamW`
- scheduler: `ReduceLROnPlateau(factor=0.5, patience=2)`
- loss: `BCEWithLogitsLoss` (use `pos_weight` if heavy imbalance)

Grid for ablation experiments:

- lr: `[1e-3, 1e-4, 5e-5]`
- backbone: `[efficientnet_b0, resnet34, resnet50]`
- image_size: `[224, 320]`
- batch_size: `[8,16,32]` (memory permitting)
- augmentation: add `RandomErasing`, MixUp, CutMix
- scheduler: `CosineAnnealingLR`, `OneCycleLR`
- optimizer: `AdamW`, `SGD(mom=0.9)`

Evaluation thresholds:

- ROC AUC for selection; for deployment choose threshold optimizing sensitivity (prefer high sensitivity in triage).

Baseline expected metrics (approximate targets)

- **Validation AUC**: 0.92 – 0.98 (varies by dataset & split)
- **Accuracy**: 0.88 – 0.95
- **Sensitivity (recall for pneumonia)**: ~0.90 target (triage focus)
  These are **benchmarks**, not guarantees — report exact numbers you obtain and include confidence intervals.

Practical tips & pitfalls

- **Data leakage**: ensure patient-level split (if dataset has multiple images per patient) to avoid inflated performance.
- **Confounders**: X-ray markers, hospital stickers may leak label; crop or mask them or use adversarial removal if observed.
- **Imbalance**: if pneumonia >> normal, use `pos_weight` in BCE or oversample minority class.
- **Explainability sanity**: verify Grad-CAM highlights pathology, not corners or markers. Perform ablation (e.g., hide lung area and observe predictions).
- **Clinical cautions**: include explicit model card and disclaimers — do NOT claim clinical use.

Extensions & advanced experiments (optional)

- Add a small segmentation model (U-Net) to extract lung ROI and feed only ROI to classifier.
- Train multi-label model for **pneumonia vs bacterial vs viral** if dataset supports.
- Add uncertainty estimation via **Deep Ensembles** or **Bayesian neural nets** for calibrated probabilities.
- Use **self-supervised pretraining** on large chest X-ray corpora (e.g., CheXpert/NL) before fine-tuning.
- Deploy TFLite or ONNX model for edge inference; measure latency.

Deliverables checklist (what to put in GitHub)

- Colab notebook (runnable top-to-bottom)
- `requirements.txt` and `environment.yml` (conda)
- `train.py`, `predict.py`, `model.py` (modular code)
- `streamlit_app.py` and Dockerfile
- `best_model.pth` (or instructions to download from artifact storage)
- Evaluation report: metrics, confusion matrices, calibration plots, Grad-CAM examples
- Model card & README with reproduction instructions and compute requirements

---

Final notes — suggested immediate next steps (do now)

1. Create GitHub repo with README and issue tracker for tasks per week.
2. Get dataset access (Kaggle or NIH) and confirm file layout.
3. Run Cells 1–11 to verify data pipeline and a small training run (1 epoch) to confirm everything works.
4. Save logs and snapshots — reproducibility is key.

# # # #

Great — below is a **practical, day-by-day sprint checklist** for the 8-week plan, broken into **six 2-hour blocks per week** (≈12 hours/week). I picked 6 blocks to match your earlier "part-time" target (10–15 hrs/week). If you prefer a true 7-day schedule or a different weekly cadence, this is easily adjustable — but I won't ask; I'll give you the full plan now.

For each 2-hour block I list precise actions, short commands/code hints where helpful, and a clear deliverable. Treat each block as a focused mini-task you can finish in ~2 hours.

Schedule assumptions (read before starting)

- Weekly cadence: **6 blocks × 2 hours = 12 hours/week** (flexible scheduling — e.g., Mon, Tue, Thu, Fri, Sat, Sun or any arrangement you prefer).
- Environment: **Google Colab (GPU)**, GitHub repo for code, Kaggle dataset (Chest X-ray Pneumonia) or NIH substitute.
- Save progress frequently to GitHub; push notebooks and scripts early. Use small commits & descriptive messages.
- Keep an issues/todo file in repo for follow-ups.

Week 1 — Project setup & data ingestion (goal: reproducible repo + EDA)

Block 1 (2 hrs) — Repo + env + dataset access

- Tasks:
  - Create GitHub repo `explainx-cxr`.
  - Add `README.md`, `.gitignore`.
  - Locally or in Colab: create `requirements.txt`.
- Commands/hints:
  - `git init`, commit README.
  - `pip install -r requirements.txt` (or in Colab, pip install libs).
- Deliverable: GitHub repo skeleton + `requirements.txt`.

Block 2 (2 hrs) — Kaggle token & dataset download

- Tasks:
  - Upload `kaggle.json` to Colab or use Kaggle API.
  - Download + unzip dataset to `data/`.
  - Verify dataset structure.
- Commands/hints:
  - `!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia`
  - `!unzip -q chest-xray-pneumonia.zip -d data/`
- Deliverable: dataset present at `data/chest_xray/...`.

Block 3 (2 hrs) — Quick file counts & patient checks

- Tasks:
  - Count images per class and per split.
  - Check for patient-level duplicates (if metadata exists) or file naming patterns.
- Code hint:
  - Small Python snippet to produce counts with `os` / `pandas`.
- Deliverable: `data_stats.csv` and short note in notebook about any data issues.

Block 4 (2 hrs) — Visual EDA (samples + anomalies)

- Tasks:
  - Display representative images (6 normal / 6 pneumonia).
  - Visually inspect for hospital markers, text overlays.
- Deliverable: EDA cell in notebook with sample images and notes on confounders.

Block 5 (2 hrs) — Basic label check & class balance plan

- Tasks:
  - Compute class imbalance metrics; decide sampling or `pos_weight` approach.
  - Draft strategy: oversample, loss weighting, or focal loss.
- Deliverable: small markdown cell in notebook describing imbalance mitigation choice.

Block 6 (2 hrs) — Create initial train/val/test split & CSV manifest

- Tasks:
  - Create stratified split (patient-aware if possible). Save CSVs `train.csv`, `val.csv`, `test.csv`.
  - Commit CSVs to repo (or store references).
- Deliverable: `data_manifests/` with CSVs.

Week 2 — Preprocessing & robust data pipeline (goal: fast & debuggable DataLoader)

Block 1 (2 hrs) — Define augmentation & transforms

- Tasks:
  - Write Albumentations transforms: train (RandomResizedCrop, flips, CLAHE), val (Resize + normalize).
  - Decide `IMG_SIZE` (224 baseline).
- Deliverable: `transforms.py` or notebook cell.

Block 2 (2 hrs) — Implement PyTorch Dataset & DataLoader

- Tasks:
  - Implement `CXRDataset` class that reads CSV manifest, applies transforms.
  - Add caching (optional) and error handling for corrupt images.
- Deliverable: `dataset.py` + test cell verifying a batch loads.

Block 3 (2 hrs) — Visualize augmentations & sanity check batches

- Tasks:
  - Plot 8 augmented samples from the same image to validate augmentations.
  - Confirm normalization values and channel order.

- Deliverable: augmentation visualization cell + notes.

Block 4 (2 hrs) — Performance tuning for DataLoader

- Tasks:
  - Tune `batch_size`, `num_workers`, `pin_memory`.
  - Try `batch_size=16` baseline; reduce to 8 if OOM.
- Deliverable: `dataloader_config.md` with optimal settings from quick tests.

Block 5 (2 hrs) — Lung ROI cropping (optional experiment)

- Tasks:
  - Implement quick lung cropping heuristic (thresholding + bounding box) OR plan to integrate U-Net later.
  - Save a small script to generate ROI crops for 500 samples for manual inspection.
- Deliverable: `preprocess_roi.py` (optional) + sample outputs.

Block 6 (2 hrs) — Save data pipeline & reproducibility notes

- Tasks:
  - Write notebook section describing preprocessing exact steps and seeds.
  - Add a small script `prepare_data.py` to re-generate manifests.
- Deliverable: `prepare_data.py` + updated README.

Week 3 — Baseline model & training loop (goal: run first meaningful training)

Block 1 (2 hrs) — Build baseline model (EfficientNet-B0 via timm)

- Tasks:
  - Create `model.py` with `build_model()` that returns backbone + head.
  - Verify model forward pass with a random batch.
- Deliverable: `model.py` + successful forward pass log.

Block 2 (2 hrs) — Define loss, optimizer, scheduler

- Tasks:
  - Use `BCEWithLogitsLoss()` with optional `pos_weight`.
  - Use `AdamW(lr=1e-4, weight_decay=1e-4)` and `ReduceLROnPlateau`.
- Deliverable: `train_config.yaml` or documented hyperparams.

Block 3 (2 hrs) — Implement training & validation loops

- Tasks:
  - Implement `train_one_epoch`, `validate`, checkpoint saving & loading.
  - Add a progress bar (tqdm) and early stopping hook.
- Deliverable: `train.py` (or notebook cells) with loops.

Block 4 (2 hrs) — Run a 1–2 epoch smoke test

- Tasks:
  - Run 1 epoch on a subset (e.g., 10% of training set) to ensure no runtime errors.
  - Fix bugs (shape errors, dtype).
- Deliverable: smoke test notebook output; commit fixes.

Block 5 (2 hrs) — Full baseline training run (start)

- Tasks:
  - Launch a full baseline run: `epochs=25`, `batch_size=16` (monitor).
  - Log training/val loss & AUC at each epoch (TensorBoard/W&B optional).
- Deliverable: baseline training started and logging enabled.

Block 6 (2 hrs) — Monitor & stabilize (early adjustments)

- Tasks:
  - Observe first few epochs: if loss diverges, reduce lr to `5e-5` or change optimizer to `SGD`.
  - Save intermediate checkpoint.
- Deliverable: stabilized run or plan to restart with tuned hyperparams.

Week 4 — Evaluation & metrics (goal: robust evaluation suite)

Block 1 (2 hrs) — Implement evaluation metrics functions

- Tasks:
  - Implement functions: `roc_auc`, `accuracy`, `f1`, `confusion_matrix`, sensitivity/specificity, precision/recall.
- Deliverable: `metrics.py`.

Block 2 (2 hrs) — Generate validation report & plots

- Tasks:
    - Compute ROC curve; plot AUC.
    - Plot confusion matrix, per-class F1.
- Deliverable: `evaluation_report.ipynb` with plots.

Block 3 (2 hrs) — Threshold tuning for deployment

- Tasks:
    - Sweep thresholds (0.1–0.9) compute sensitivity/specificity; choose threshold prioritizing sensitivity (triage).
- Deliverable: threshold table & chosen operating point.

Block 4 (2 hrs) — Calibration checks

- Tasks:
    - Compute calibration curve (reliability diagram).
    - Try temperature scaling (simple post-hoc calibration) if miscalibrated.
- Deliverable: calibration plot and post-hoc calibration script.

Block 5 (2 hrs) — Analyze failure cases

- Tasks:
    - Sample false positives & false negatives; visually inspect Grad-CAM later.
    - Document common failure modes (low contrast, label noise, markers).
- Deliverable: `failure_cases/` folder with 20 annotated examples.

Block 6 (2 hrs) — Update README with evaluation summary

- Tasks:
    - Add a summary of baseline metrics and decisions based on metrics.
- Deliverable: updated `README.md` evaluation section.

Week 5 — Explainability & uncertainty (goal: Grad-CAM + MC-Dropout integrated)

Block 1 (2 hrs) — Integrate Grad-CAM

- Tasks:
    - Add `explainability.py` using `pytorch-grad-cam`.
    - Test on 10 validation images and save visualizations.
- Deliverable: Grad-CAM script + sample images.

Block 2 (2 hrs) — Grad-CAM sanity checks

- Tasks:
  - Check whether CAM highlights lungs/pathology — if not, check for confounders (borders).
  - Run ablation: blank out high-priority regions and see prediction change.
- Deliverable: sanity check notes & example cases.

Block 3 (2 hrs) — Implement MC-Dropout

- Tasks:
  - Turn on dropout at inference (function to enable Dropout modules).
  - Run T=30 forward passes for sample images; compute mean & std.
- Deliverable: `uncertainty.py` and per-image mean/std table.

Block 4 (2 hrs) — Produce combined visual outputs

- Tasks:
  - For each sample: show original image, Grad-CAM overlay, predicted prob, uncertainty std, and thresholded decision.
  - Save a grid of 12 combined examples.
- Deliverable: `explainability_report.pdf` or markdown with images.

Block 5 (2 hrs) — Evaluate uncertainty usefulness

- Tasks:
  - Check whether high-uncertainty correlates with errors (compute AUROC of uncertainty as error detector).
- Deliverable: short analysis table and conclusions.

Block 6 (2 hrs) — Integrate explainability into inference pipeline

- Tasks:
  - Add `predict_with_explainability()` that returns (prob, std, CAM_image).
  - Test integration end-to-end.
- Deliverable: single CLI/Notebook function for inference+CAM.

Week 6 — Model improvements & ablation (goal: improve final model with experiments)

Block 1 (2 hrs) — Plan experiment grid & logging

- Tasks:
  - Create experiment matrix (backbones, augmentations, lr, img_size).
  - Prioritize quick wins: `EfficientNetB0 → ResNet34`, add `RandomErasing`, `MixUp`.
- Deliverable: `experiments.csv` and run plan.

Block 2 (2 hrs) — Run augmentation experiments

- Tasks:
  - Run 2–3 short jobs: with/without `RandomErasing`, with `CLAHE`, MixUp.
  - Compare val AUC after 5 epochs.
- Deliverable: augmentation comparison summary.

Block 3 (2 hrs) — Backbone & input size experiment

- Tasks:
  - Try `resnet34` at 224 and `efficientnet_b0` at 320.
  - Monitor memory and epoch times; note best accuracy/time tradeoff.
- Deliverable: backbone comparison summary.

Block 4 (2 hrs) — Regularization & optimizer experiments

- Tasks:
  - Try `weight_decay` changes, `AdamW` vs `SGD` with momentum, and label smoothing (if desired).
- Deliverable: small table with results.

Block 5 (2 hrs) — Ensembling / model averaging (optional)

- Tasks:
  - If two models complement each other, test simple ensemble (average probabilities).
  - Evaluate ensemble AUC gain.
- Deliverable: ensemble evaluation and decision whether to include.

Block 6 (2 hrs) — Select final model & freeze config

- Tasks:
  - Based on experiments, choose the final model and document final hyperparameters.
  - Save final model weights as `best_model_final.pth`.
- Deliverable: `final_config.yaml` and saved weights.

Week 7 — Deployment & demo (goal: working Streamlit demo + container)

Block 1 (2 hrs) — Export model (TorchScript / ONNX)

- Tasks:
  - Trace & save `model_traced.pt` or export ONNX.
  - Test a few inference calls using exported artifact.
- Deliverable: `model_traced.pt` or `model.onnx`.

Block 2 (2 hrs) — Build Streamlit app skeleton

- Tasks:
  - Create `streamlit_app.py` that loads model, accepts upload, shows prob + CAM + uncertainty.
  - Test locally in Colab with `ngrok` or locally if you run locally.
- Deliverable: working minimal Streamlit app.

Block 3 (2 hrs) — Integrate UI polish & error handling

- Tasks:
  - Add progress/spinner, descriptive text, and display for uncertainty/confidence.
  - Limit file size and show helpful error messages for non-Xray images.
- Deliverable: polished Streamlit UI.

Block 4 (2 hrs) — Create Dockerfile & local container test

- Tasks:
  - Write `Dockerfile` for app.
  - Build locally or in cloud runner; verify that the container runs and predicts.
- Deliverable: `Dockerfile` and instructions.

Block 5 (2 hrs) — (Optional) Deploy to Cloud Run / Heroku / Streamlit Cloud

- Tasks:
  - Follow small deployment checklist for chosen provider; push container or app.
  - Secure app (disable public posting of model weights).
- Deliverable: deployed demo URL (if you deploy) or deployment README.

Block 6 (2 hrs) — Security, privacy & model card for demo

- Tasks:
    - Add explicit disclaimer text in the app (not for clinical use).
    - Ensure uploaded files are not stored permanently.
- Deliverable: updated Streamlit app with model card link.

Week 8 — Report, README, final polish & wrap-up (goal: production-ready repo + deliverables)

Block 1 (2 hrs) — Final evaluation & figures consolidation

- Tasks:
    - Recompute final metrics on test set; produce final ROC, confusion matrix, and calibration.
    - Gather Grad-CAM examples (TP/FP/FN).
- Deliverable: `final_evaluation.pdf` or notebook.

Block 2 (2 hrs) — Write model card & limitations

- Tasks:
    - Fill out Model Card: intended use, training data, evaluation, limitations, ethical considerations.
- Deliverable: `MODEL_CARD.md`.

Block 3 (2 hrs) — Final README & reproduction guide

- Tasks:
    - Write step-by-step reproduction: Colab notebook link, how to run training/inference, compute needs.
- Deliverable: polished `README.md`.

Block 4 (2 hrs) — Prepare slide deck / video demo

- Tasks:
    - Create a short slide deck (6–8 slides) covering problem, dataset, model, results, demo, limitations.
    - Optionally record a 3–5 minute demo video showing the Streamlit app.
- Deliverable: `presentation.pdf` and optional video file.

Block 5 (2 hrs) — Clean code & final commits

- Tasks:
  - Refactor messy notebook cells into `train.py`, `eval.py`, `predict.py`, `explain.py`.
  - Add `requirements.txt` and environment notes.
- Deliverable: codebase tidy, tests pass.

Block 6 (2 hrs) — Release & outreach

- Tasks:
  - Create a GitHub release with tags, upload model artifact or provide download link.
  - Write a short GitHub Issue with future work and next steps; optionally post to LinkedIn/GitHub with project summary.
- Deliverable: GitHub release + public repo ready for recruiters/interviews.

Quick productivity & quality rules (follow during the sprint)

- Commit early and often — one logical change per commit.
- Use clear issue titles and a kanban (To Do / In Progress / Done) in GitHub Projects or a simple spreadsheet.
- Keep a `RESULTS.md` recording each experiment's config and metrics (backbone, lr, augment, val AUC).
- When running long experiments, keep logs (TensorBoard/W&B) and snapshots of best checkpoints.
- Immediately document any data leakage or dataset issues in `DATA_NOTES.md`.
- For any time you feel stuck: run a tiny end-to-end pipeline on a 200-image subset to debug.

Final checklist before submission / demo

- ✅ Notebook runs top-to-bottom in Colab (or has explicit cells to run in order).
- ✅ Final model weights and exported artifact included or linked.
- ✅ Streamlit demo works and includes clear disclaimer.

- ✅ Model card & README with instructions and limitations.
- ✅ Short slide deck and/or demo video.
- ✅ `RESULTS.md` with experiments and final selected configuration.